

Design and Fabrication of a Biomimetic Snake Robot on a Mecanum-Wheel Base

Haonan Li

h.li0220252@arts.ac.uk

MSc Creative Robotics, Creative Computing Institute,
University of the Arts London
London, United Kingdom

Tanyang Li

t.li0620251@arts.ac.uk

MSc Creative Robotics, Creative Computing Institute,
University of the Arts London
London, United Kingdom

Abstract

This paper describes a biomimetic snake robot built for the *1001 Nights* project, where it was intended to support the story of the Traveler and the Snake. The robot combines a four-wheel Mecanum chassis, a reduced robotic-arm head, and an eight-servo articulated tail. Rather than attempting full biological realism, the project breaks snake-like motion into three controllable behaviours: mobile positioning, head gesture, and a travelling wave along the tail. The final system uses two SN754410NE motor drivers for the four N20 wheel motors, a PCA9685 PWM board for servo control, and a Python-to-Arduino serial bridge for Xbox controller input. The build process exposed practical limits in 3D printed tolerances, current draw, wiring density, and controller reliability. Those failures shaped the final design more than the original sketch did. For a stage robot, repeatable and safe motion mattered more than adding extra functions that the power system could not support.

CCS Concepts

• **Computer systems organization** → **Robotic control**; • **Hardware** → **Electromechanical systems**; • **Applied computing** → *Performing arts*.

Keywords

biomimetic robot, snake robot, Mecanum wheel, servo control, creative robotics, physical computing

1 Introduction

The project began with a simple question from an earlier robotics exercise: what would change if a robotic arm were no longer fixed to a table, but mounted on a base that could move? In the *1001 Nights* project, that idea became a mechanical snake for the Traveler and the Snake story. The base provides stage movement, the arm becomes an expressive head, and the tail gives the machine a visible animal motion.

The goal was not to copy a real snake. A real snake uses distributed contact, friction, and body deformation in ways that are difficult to reproduce in a compact student prototype. We treated the snake as a set of cues that could be engineered: a curved body, a head that searches or reacts, and a wave that travels along the tail. This framing made the robot feasible with the parts available in the studio.

Early concepts leaned toward a toy-like stage prop. One version imagined a small vehicle dragging a group of decorative snakes. After the first review, the design moved away from passive dragging. We considered a trailer-like tail connected with elastic cord, but

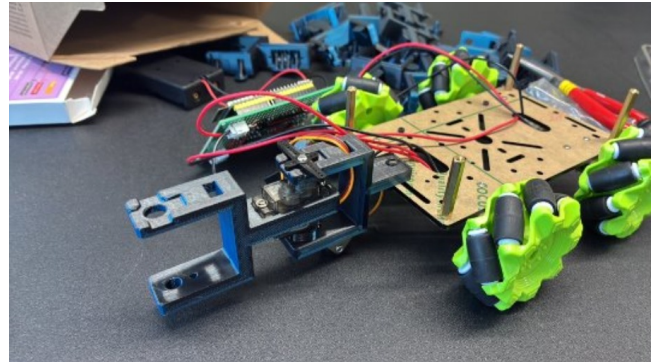


Figure 1: Final prototype with Mecanum base, robotic-arm head, and articulated servo tail.

the final prototype uses servo joints so that the tail can be driven directly and tuned in code.

2 System overview

The final robot has four main subsystems: the mobile base, the robotic-arm head, the articulated tail, and the power and wiring system. Table 1 summarises the hardware and the main design tradeoffs.

The chassis gives the robot its stage position. The robotic arm carries the head gesture. The tail runs a continuous wave so that the body does not look like a static trailer. Arduino handles the main control loop, the PCA9685 board generates servo PWM signals, and the SN754410NE driver boards handle the four DC motors.

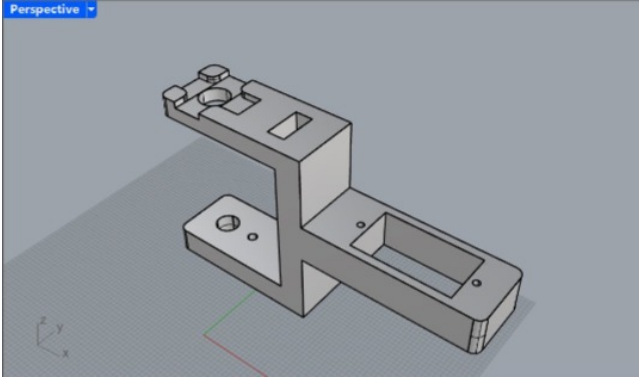
3 Robotic-arm head

The head mechanism came from a previous robotic-arm exercise. In that earlier build, ultrasonic sensors around the base detected objects and made the arm turn toward them. That version also revealed a mechanical limit: the arm could not rotate freely through 360 degrees because one side had to be reinforced to balance the upper structure.

Mounting the arm on a mobile base changed its role. It no longer needed to work as a full manipulator. It needed to read as a snake head. For that reason, the final build removed the bottom rotation stage and kept the upper arm sections, which were more useful for posture. This reduced mass at the front of the chassis and made the head easier to include in the overall body.

Table 1: Main subsystems in the final prototype.

Subsystem	Hardware	Role	Design decision
Robotic-arm head	Modified arm, MG996R-class servos, PCA9685 channels 8 and 9	Head posture and visible gesture	The lower rotation stage was removed to reduce size and weight. The final code keeps two manually controlled head servos.
Servo tail	Eight MF90 servos, 3D printed connectors, small support wheels	Travelling body wave	Active servo joints replaced the elastic and trailer concepts. This made the motion easier to tune.
Mecanum base	Four Mecanum wheels, four N20 motors, two SN754410NE driver ICs	Forward motion, lateral movement, and turning on stage	The ESP32 version was abandoned after overheating. Arduino was more reliable for the demonstration.
Power and wiring	7.4 V battery, 5 V supply, controller boards, wiring harnesses	Motor, logic, and servo supply	Future work should separate the motor, logic, and servo rails more cleanly.

**Figure 2: CAD iteration for the tail connector, including the servo pocket, linkage area, and support wheel position.**

4 Tail mechanics

The tail is the clearest biomimetic feature in the prototype. We tested several ideas before settling on servo joints. A bicycle-like steering linkage was discussed, and an elastic trailer structure was also considered. The servo solution was simpler to control because each joint has a target angle. It also made the wave visible enough for a stage audience.

The first 3D printed model already included the important features: a servo pocket, a servo horn area, and a mounting point for a small support wheel. It did not leave enough tolerance for the real printed parts, so assembly was awkward. The second version corrected some dimensions but allowed the top horn connection to slip during motion. The third version used more careful measurements for the servo body, locking holes, tail connector, and chassis attachment. That version became the working tail.

5 Serpentine gait and servo mapping

The tail gait uses a phase-shifted sine wave. Each servo is treated as one joint along the body. A phase delay between neighbouring joints makes the curve travel along the tail:

```
const int numServos = 8;
float amplitude = 35.0;
float speed = 0.05;
float phaseShift = M_PI / 4.0;
float offsetAngle = 90.0;
```

Figure 3: Tested control parameters for the tail wave: servo count, amplitude, speed, phase shift, and centre angle.**Table 2: PCA9685 channel mapping in the final Arduino code.**

Channels	Device	Control role
0 to 7	Eight MF90 tail servos	Updated by updateSnake() using Equation 1
8	Head base servo	Manual motion from Xbox right-stick X.
9	Head end servo	Manual motion from Xbox right-stick Y.
10 to 15	Reserved	Future gripper, lighting, or sensing actions.

$$\theta_i(t) = A \sin(\omega t - i\Delta\phi) + \theta_{\text{offset}}. \quad (1)$$

Here, i is the joint index, A is the swing amplitude, ω is the angular frequency, $\Delta\phi$ is the phase shift between joints, and θ_{offset} is the centre angle. In the tested code, the tail used eight servos, an amplitude of 35° , a phase shift of $\pi/4$, and a centre angle of 90° .

The Mecanum base already supplies the forward force, so the tail does not need to propel the robot by itself. Its job is mostly expressive. The wave makes the body look alive while the wheels move the robot across the stage.

6 Control software

The final control software is split between a computer and the Arduino. A Python bridge reads an Xbox controller through pygame,

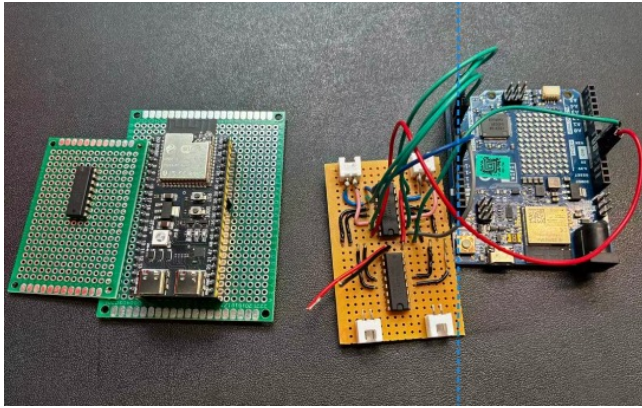


Figure 4: Controller-board comparison during the ESP32 and Arduino tests.

fixes the common Windows axis mapping problem, and sends joystick values to the Arduino over serial using `pyserial`. The Arduino maintains the tail wave, parses the newest serial packet, applies a timeout safety stop, and updates the two head servos.

The packet format is deliberately small:

```
J,lx,ly,rx,ry\n
```

Each axis is an integer from -100 to 100 . The left stick controls the Mecanum base. The right stick controls the two head servos. The Python bridge sends a packet every 0.04 seconds, which is about 25 packets per second. A dead zone removes small stick noise.

The Arduino loop is short enough to inspect directly:

```
void loop() {
  updateSnake();
  handleSerialInput();
  updateXboxTimeout();
  updateManualArmFromJoystick();
}
```

The final code removes the earlier automatic arm up and down sequence. The head is controlled manually instead. That proved more useful for a stage demonstration because the operator can adjust the head in response to the scene while the tail continues its wave.

7 Mecanum chassis and electronics

The mobile base started with four Mecanum wheels and four N20 motors. Since the kit did not include a motor controller board, we built one. Each SN754410NE chip can drive two motors, so two chips were enough for the four-wheel base.

The circuit was tested first on a breadboard, then rebuilt on perfboard for Arduino. We later tried an ESP32 version to reduce wiring and leave room for more features. That version overheated during wiring and upload tests, which suggested an unsafe current path or overloaded rail. For the final demonstration, the Arduino version was safer and easier to debug.

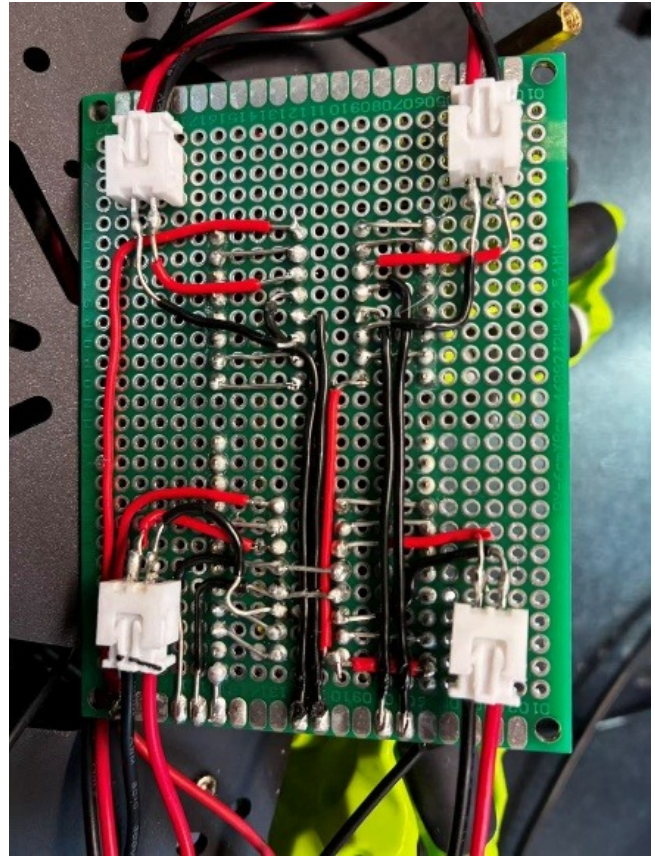


Figure 5: Hand-soldered motor driver board used in the chassis tests.

8 Failure-driven design decisions

Several failures set the practical limits of the robot. Low voltage made motors and servos unstable under load. The handmade PCB reduced cable clutter but made faults harder to isolate. The damaged ESP32 made the team return to Arduino for the final demonstration. The servo current also forced a more conservative approach: motor power, logic power, and servo power should not be treated as one simple rail.

The main lesson was simple: a stage robot must repeat its motion reliably. A system with more pins or more planned features is not better if it is difficult to power safely.

9 Team contributions

Haonan Li focused on early tail structure, electronics fabrication, and the final control interface. He produced the first tail model, turning the servo pocket, horn clearance, and support wheel position into a printable form. He also handled much of the motor-driver soldering and board integration, including the SN754410NE driver board, connector layout, and cable fixing inside the chassis. On the control side, he moved the project beyond the first keyboard commands by building the Python and controller workflow that made the robot easier to operate during the demonstration.

Table 3: Software structure used in the final prototype.

Code component	Main function	Implementation detail
Python Xbox bridge	Reads the controller and sends serial packets to Arduino.	Runs at 115200 baud. Uses a 0.08 dead zone. Supports Windows and standard axis mappings.
Arduino controller	Controls four N20 motors, eight tail servos, and two head servos.	Uses two SN754410NE motor drivers and the PCA9685 PWM board. Keeps the original w/s/a/d/x serial commands.
Motion mixing	Converts joystick values into wheel PWM and head-servo targets.	Left-stick X and Y are mixed for Mecanum motion. Right-stick X and Y step the head servos while the stick is held.

Table 4: Failures and design decisions.

Problem	Observation	Design decision
Low voltage under load	Motors and servos became unstable.	Size the supply from real current draw, not only from component labels.
Hard-to-debug wiring	The integrated board reduced cables but hid some faults.	Test modules separately before soldering them into one board.
ESP32 overheating	The board became hot after power was applied.	Use Arduino for the demonstration because it was more predictable.
Servo current peaks	Several servos could move at once.	Separate motor, logic, and servo power planning in the next version.

Tanyang Li focused on tail iteration, 3D printing, gait code, and system debugging. After the first printed model exposed assembly errors, he remeasured the servos, locking holes, connector geometry, and chassis interface, then updated and printed later versions of the tail. He wrote and tested the tail-servo control code, including the servo-centering script and the phase-shifted sine wave. He also worked on the first keyboard control scheme and helped debug wiring, overheating, ESP32 risk, and possible power backflow.

Both authors assembled and tested the robotic-arm head, integrated the PCA9685 servo control, and tuned the combined motion of the Mecanum base, head, and tail. The final stage behaviour came from tradeoffs made during integration: which servos to keep active, which gestures to simplify, which control board to trust, and how much movement the power system could support safely.

10 Outcome and future work

The prototype can move, hold a head posture, and run a visible tail wave. It brings together the four-wheel Mecanum base, reduced robotic-arm head, eight-servo tail, PCA9685 servo control, and phase-shifted gait into one working body.

The next version should focus less on adding more channels and more on reliability. The robot still depends on separate power sources, which limits true mobile performance. A battery pack with suitable discharge capacity would make the system cleaner, but it

must be chosen with weight and safety in mind. The head can also shift the centre of mass forward, so a rear counterweight or stricter head-angle limits may be needed.

Closed-loop sensing is the most useful next step. An IMU, distance sensor, or proximity sensor could let the snake change head posture or tail motion in response to the scene. The tail wave should also be linked more tightly to the Mecanum direction and speed so that the whole robot reads as one body rather than a vehicle pulling a moving tail.

11 Reflection

The project was useful because it connected mechanical design, printing tolerance, motor driving, servo timing, wiring, debugging, and stage behaviour in one build. The tail parts taught us how much printed tolerances matter. The handmade controller board made current paths and connector choices less abstract. The damaged ESP32 and overheated wiring were frustrating, but they made the final design safer.

The next version should move from “it moves” to “it moves reliably and reacts to the scene”. That means cleaner power, better balance, sensing, and wiring that can be repaired without taking the whole body apart.